



Seara Data

Seara, Menemani Perjalanan
Datamu Lebih Terarah.

Portfolio Bootcamp Data Analyst – Python

Batch 2

By Rahmanida S

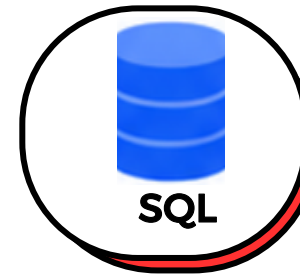
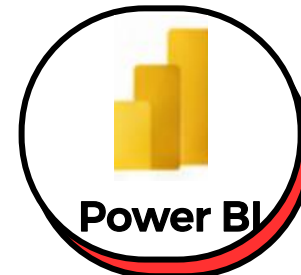
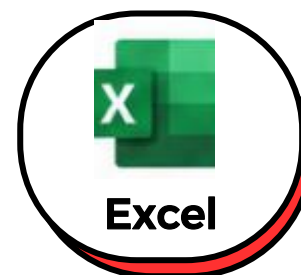




**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

Bootcamp Data Analyst

**Tools Wajib Data
Analyst**



4 Weeks Intensive Bootcamp



About Me

As a Chemistry graduate and an experienced QA/QC Supervisor with a four-year track record in manufacturing, I focus on driving process optimization and problem-solving. In today's landscape, I observe that the most impactful operational decisions are entirely data-driven. This insight inspired me to develop expertise in data analytics. To leverage this expertise and expand my career reach toward strategic analytical roles, I am committed to building new technical capabilities through this Data Analyst Bootcamp.

Positioning myself for a strategic transition into data analytics, I am continuously building technical skills in advanced data processing tools. I aim to merge my industrial experience with these new frameworks to architect smarter, high-impact business decisions.



Rahmanida S

PESERTA BOOTCAMP

**BOOTCAMP DATA ANALYST
BY SEARA DATA (BATCH 2)**



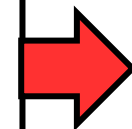
Data Analyst Learning Journey

Seara, Menemani Perjalanan
Datamu Lebih Terarah.

Excel

Understand & Prepare Data
Become a Data Analyst
Microsoft Excel Basic
Microsoft Excel Intermediate
Power Query

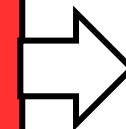
Output: Clean dataset ready for analysis.



Power BI

Analyze & Visualize Data
Microsoft Power BI Basic
DAX & Data Calculation
Microsoft Power BI Advanced
Visualization & Interactive Report

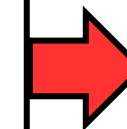
Output: Interactive dashboard and business insights.



Python

Transform & Automate Data
Python Basic
Data Cleaning & Formatting
Python Advanced
Data Transformation

Output: Processed data pipeline and efficient workflows.



SQL

Query & Extract Insights
SQL Basic
Query & Filter Data
SQL Intermediate
Join, Aggregation & Subquery

Output: Insights directly from databases.



Summary of Python Session

Seara, Menemani Perjalanan
Datamu Lebih Terarah.

What I Learned

1 EXTRACT & DATA UNDERSTANDING

(Extract and understanding structure of data before analysis)



Load Data



Explore Data



Validate Data



Understanding

Basic Query

- | | |
|--------------------------------------|--|
| • <code>df = pd.read_csv()</code> | <code>#Load dataset</code> |
| • <code>df.shape</code> | <code># Numbers of rows & columns</code> |
| • <code>df.head()</code> | <code># Preview data</code> |
| • <code>df.columns</code> | <code># List columns</code> |
| • <code>df.info() / df.dtypes</code> | <code># Data types</code> |
| • <code>df.isna().sum()</code> | <code># Missing value</code> |
| • <code>df.duplicated().sum()</code> | <code># Duplicates</code> |
| • <code>df.value_counts()</code> | <code># Check categories</code> |

Dataset



Customer



Product



Store



Transaction

Key Info

Before cleaning or analyzing, we need to know what the data contains, whether it is reliable, and what each row represents.



Summary of Python Session

Seara, Menemani Perjalanan
Datamu Lebih Terarah.

What I Learned

2 TRANSFORM, CLEAN & MERGE

(Make sure the data is valid, konsisten, and ready to analysis)

TRANSFORM & CLEANING

`df.fillna()` → handle missing values

`df.drop_duplicates()` → remove duplicate rows

`df.astype()` → change data type

`pd.to_numeric()` → Convert values to numeric

`pd.to_datetime()` → Convert values to datetime

`df.replace()` → Replace inconsistent values

`df.str.strip()` → Remove leading/trailing spaces

`df.describe()` → Check numerical distribution

`df[condition]` → Filter data based on conditions



Summary of Python Session

Seara, Menemani Perjalanan
Datamu Lebih Terarah.

What I Learned

2 TRANSFORM, CLEAN & MERGE

(Make sure the data is valid, consistent, and ready to analysis)

MERGE DATA

`df.merge()` → Join tables using key columns

`pd.concat()` → Combine data by rows or columns

DATA VALIDATION

`df.shape` → Check rows and columns

`df.info()` → Check structure and data types

`df.isnull().sum()` → Count missing values

`df.duplicated().sum()` → Count duplicate rows

`df.value_counts()` → Check category frequency

`df.nunique()` → Count unique values



Summary of Python Session

Seara, Menemani Perjalanan
Datamu Lebih Terarah.

What I Learned

3

ANALYZE, VISUALIZE & INSIGHT

(Translated the clean data to business insight)

1

CREATE BUSINESS METRICS

METRIC	FORMULA
Net Sales	$qty \times unit_price \times (1 - discount\%)$
COGS	$qty \times cost_price$
Gross Profit	$net_sales - COGS$
Profit After Shipping	$net_sales - COGS - shipping_cost$
Margin %	$gross_profit / net_sales \times 100$

2

AGGREGATE & COMPARE

GROUPBY

```
df.groupby('dimension').agg(  
    metric1=('value', 'sum'),  
    metric2=('value', 'mean'),  
    metric3=('value', 'nunique')  
)
```

PIVOT (COMPARISON)

```
pd.pivot_table(  
    df,  
    index='dimension',  
    columns='comparsion',  
    values='value',  
    aggfunc='sum'  
)
```

DIMENSIONS ANALYSIS

Quarter

Category

Channel

Discount



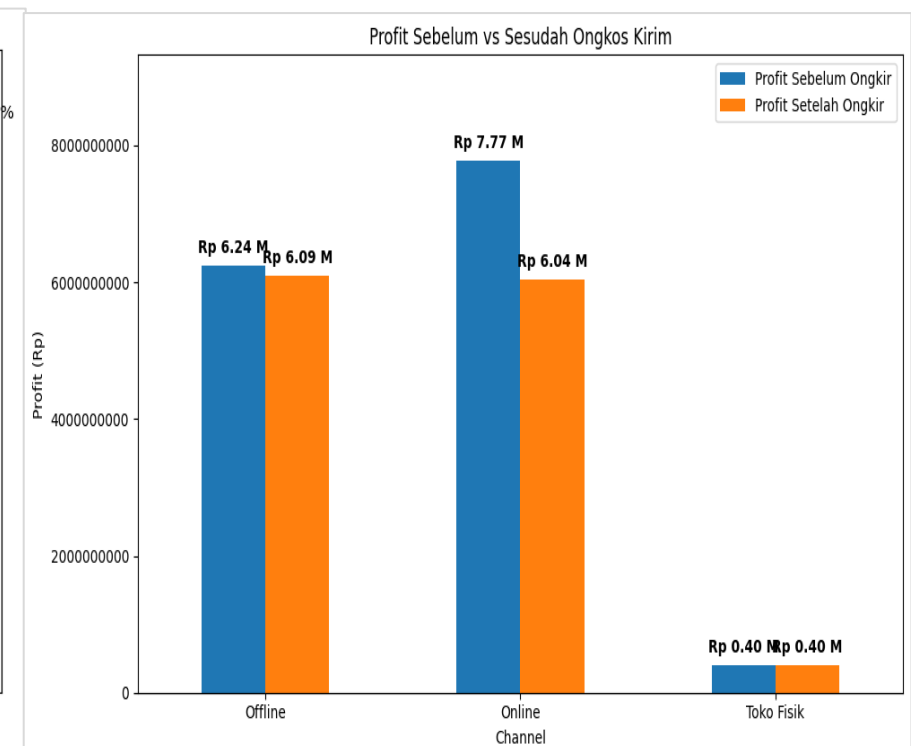
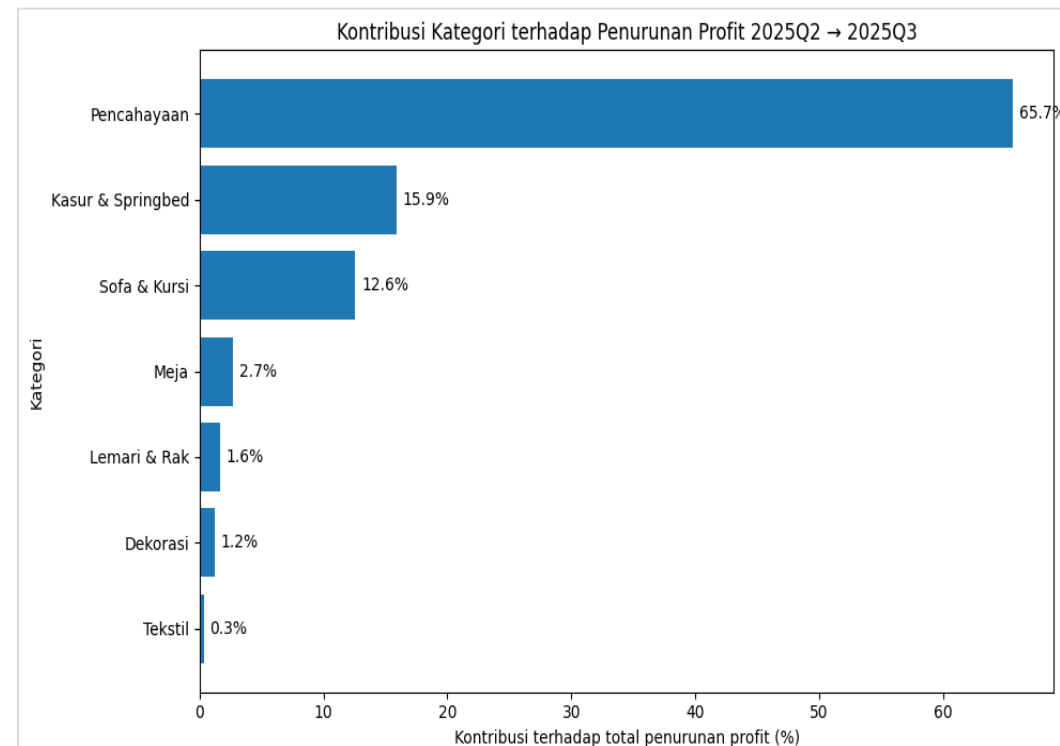
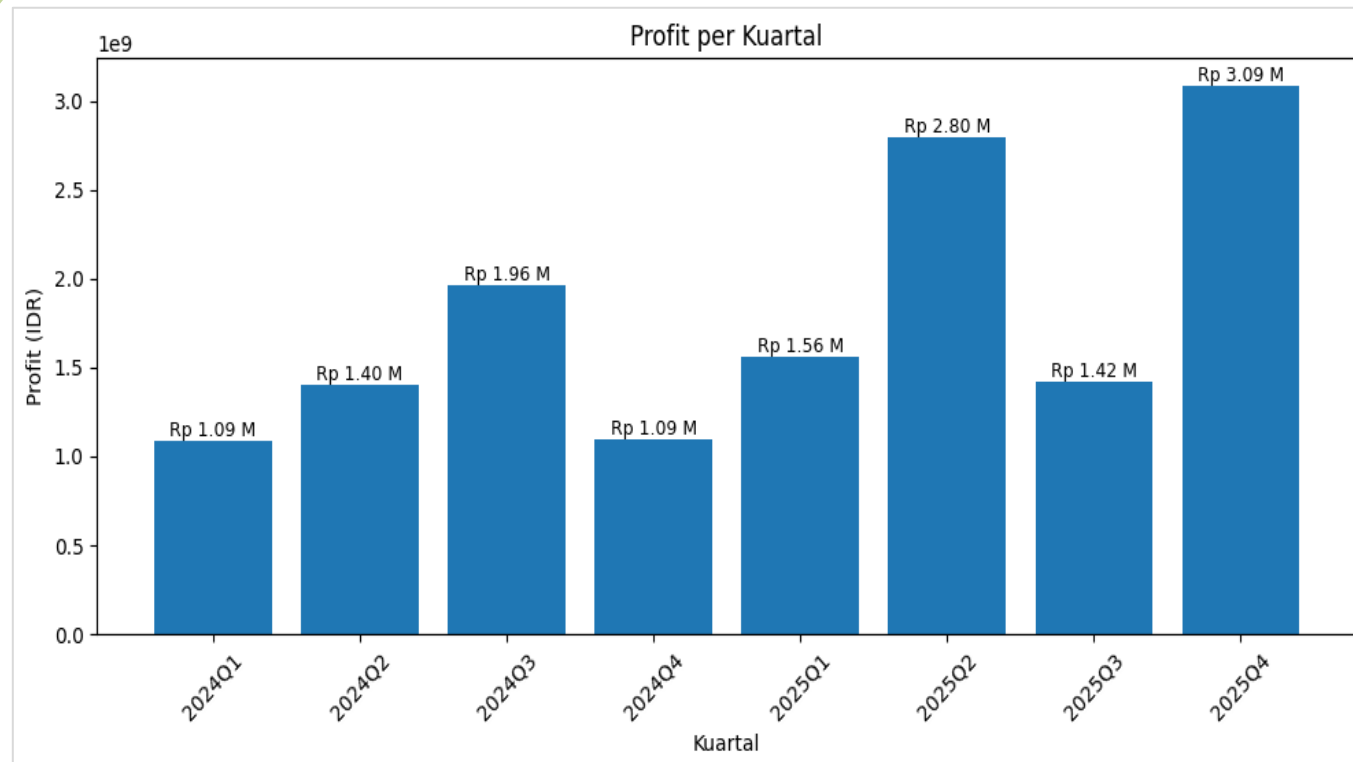
Summary of Python Session

What I Learned

Seara, Menemani Perjalanan
Datamu Lebih Terarah.

3 ANALYZE, VISUALIZE & INSIGHT (Translated the clean data to business insight)

3 VISUALIZATION



Lampiran

- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub

Documentations



**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

#Task

- 1 Berapa besar penurunan laba dan di kuartal mana persisnya
- 2 Apakah penyebabnya bauran produk, kanal penjualan, atau kebijakan diskon
- 3 Kategori mana yang paling terdampak dan seberapa besar kontribusinya
- 4 Apakah kanal online tetap menguntungkan setelah ongkos kirim dihitung
- 5 Tindakan apa yang paling cepat memulihkan margin tanpa membunuh volume

Lampiran

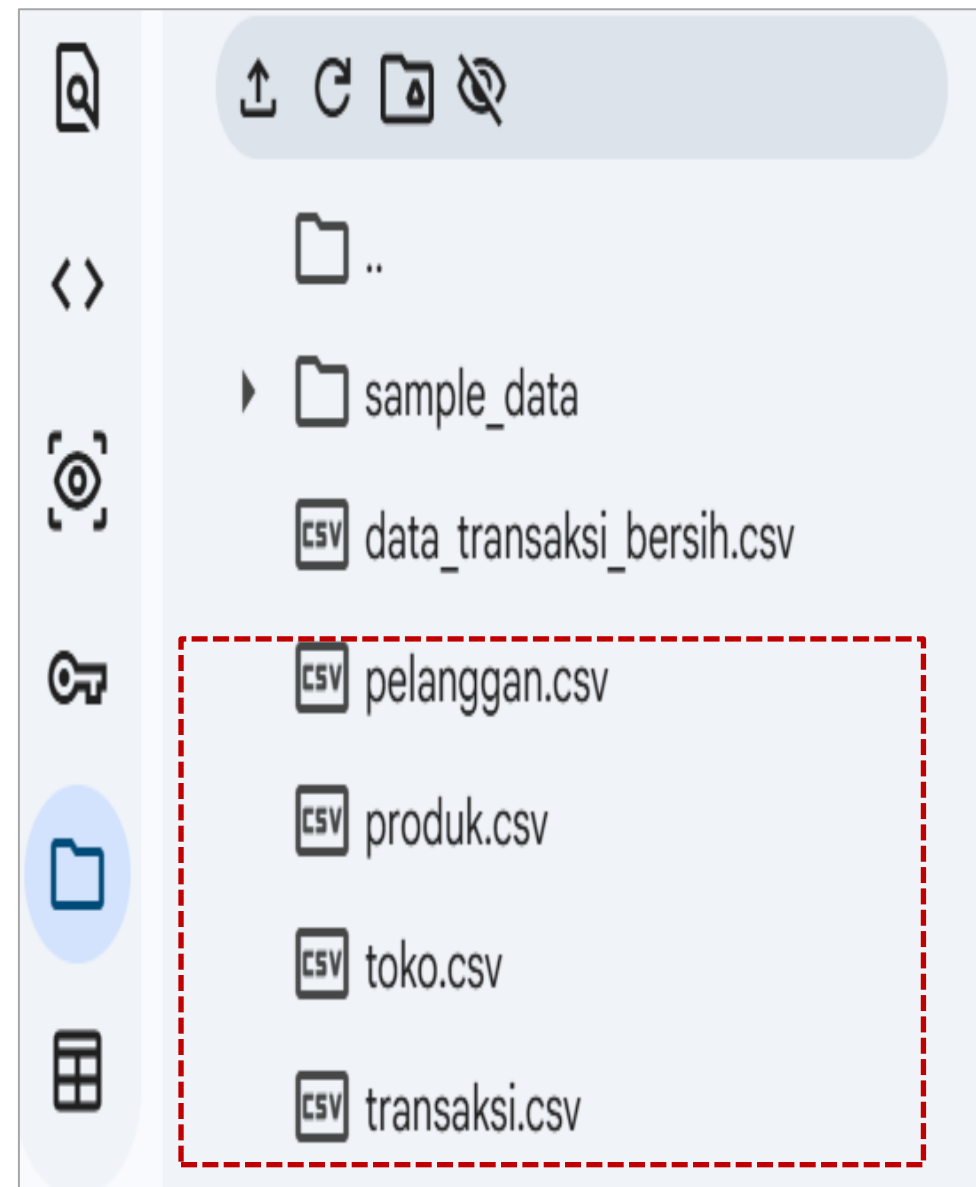
- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub

Documentations & Script



**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

1. Import & Extract Data



Load/Upload data to Google Collabs

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Import Pandas, Numpy, Matplotlib and Seaborn

```
cust=pd.read_csv('pelanggan.csv')
prod=pd.read_csv('produk.csv')
store=pd.read_csv('toko.csv')
trans=pd.read_csv('transaksi.csv')
```

Read Data using Pandas

Lampiran

- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub

Documentations & Script



**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

2. Data Validation Before Analysis

- Dataset Checking & Results

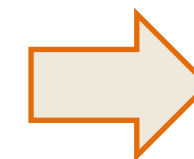
```
print("Customer :", cust.shape)
print("Product  :", prod.shape)
print("Store    :", store.shape)
print("Trans     :", trans.shape)
```



```
Customer : (6825, 8)
Product  : (170, 7)
Store    : (13, 6)
Trans     : (12240, 12)
```

- Preview Data

```
print("CUSTOMER")
display(cust.head())
```



	customer_id	customer_name	gender	birth_year	city	province	join_date	membership
0	C20000	Hendra Ramadhan	Wanita	2000	Yogyakarta	DI Yogyakarta	2023-04-25	Basic
1	C20001	Fitri Ramadhan	Wanita	1994	Yogyakarta	DI Yogyakarta	2023-12-20	NaN

- Preview Column Name

```
print("Customer:", cust.columns.tolist())
print("Product:", prod.columns.tolist())
print("Store:", store.columns.tolist())
print("Trans:", trans.columns.tolist())
```

```
Customer: ['customer_id', 'customer_name', 'gender', 'birth_year', 'city', 'province', 'join_date', 'membership']
Product:  ['product_id', 'product_name', 'category', 'sub_category', 'brand', 'list_price', 'cost_price']
Store:    ['store_id', 'store_name', 'city', 'province', 'store_type', 'open_date']
Trans:    ['order_id', 'order_date', 'customer_id', 'product_id', 'store_id', 'channel', 'qty', 'unit_price', 'discount_pct', 'sh
```

Lampiran

- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub

Documentations & Script



- Structure Data Info

```
print("=== CUSTOMER ===")
cust.info()
```



```
=== CUSTOMER ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6825 entries, 0 to 6824
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customer_id     6825 non-null  object
1   customer_name   6825 non-null  object
2   gender          6399 non-null  object
3   birth_year      6825 non-null  int64
4   city            6825 non-null  object
5   province        6825 non-null  object
6   join_date       6825 non-null  object
7   membership      6525 non-null  object
dtypes: int64(1), object(7)
memory usage: 426.7+ KB
```

**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

3. Data Understanding

Dataset	Column	Problem	Qty
Customer	gender	Missing Value	426
Customer	membership	Missing Value	300
Customer	join_date	Datetime (Still object)	1
Product	brand	Missing Value	6
Product	cost_price	Missing Value	14
Store	open_date	Datetime (Still object)	1
Transaction	store_id	Missing Value	245
Transaction	discount_pct	Missing Value	852
Transaction	shipping_cost	Missing Value	586
Transaction	payment_method	Missing Value	356
Transaction	order_date	Datetime (Still object)	1
Transaction	unit_price	Data Type (Still object)	1

Lampiran

- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub

Documentations & Script



**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

4. Data Cleaning (For Data “Pelanggan”)

4.1 Duplicates Data

```
#. Cek Duplicate Customer  
cust.duplicated().sum()  
  
np.int64(25)  
  
# Cek duplicated menggunakan customer_id.  
cust['customer_id'].duplicated().sum()  
  
np.int64(25)
```

Problems:

- Duplicates date : 25



```
#.Hasilnya sama persis dengan customer_id, jadi bisa di hapus dengan:  
cust=cust.drop_duplicates()  
  
#. Cek lagi jumlah duplicatednya  
cust.duplicated().sum()  
  
np.int64(0)
```

Solutions :

- Check duplicates data based on customer_id.
- The data check showed that both are the same, so one of them can be removed.

Lampiran

- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub

Documentations & Script



**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

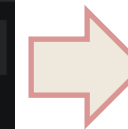
4. Data Cleaning (For Data “Pelanggan”) ”

4.1 Missing Value

```
cust.isnull().sum()
[17]
...
customer_id    0
customer_name  0
gender        426
birth_year     0
city           0
province       0
join_date      0
membership     300
dtype: int64
```

```
missing_cust = pd.DataFrame({
    'missing_count': cust.isnull().sum(),
    'missing_pct': cust.isnull().mean()
})
missing_cust
```

	missing_count	missing_pct
customer_id	0	0.000000
customer_name	0	0.000000
gender	426	6.241758
birth_year	0	0.000000
city	0	0.000000
province	0	0.000000
join_date	0	0.000000
membership	300	4.395604



```
# Missing Value Gender, Ubah Jadi "Unknown"
cust['gender'] = cust['gender'].fillna('Unknown')

# Pengecekan hasil Missing Value Gender
cust['gender'].value_counts(dropna=False)
```

	count
gender	
Wanita	3388
Pria	2987
Unknown	425

dtype: int64

Problems:

- Missing Value for Gender Column : 426

Solutions :

- Check the presentation for decision.
- The “Null” data will be replace with Unknown

Lampiran

- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub

Documentations & Script



Seara, Menemani Perjalanan
Datamu Lebih Terarah.

4. Data Cleaning (For Data “Pelanggan”)

4.1 Types of Data

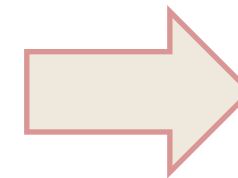
```
#. Cek Data Type
cust.dtypes
```

	0
customer_id	object
customer_name	object
gender	object
birth_year	int64
city	object
province	object
join_date	object
membership	object

dtype: object

Problems:

- Join Date format : Object



```
#. Format Date pada Kolom Join Date masih Object.
cust['join_date']=pd.to_datetime(cust['join_date'])
```

```
# Cek data Type
cust.dtypes
```

	0
customer_id	object
customer_name	object
gender	object
birth_year	int64
city	object
province	object
join_date	datetime64[ns]
membership	object

dtype: object

Solutions :

- Replace to Datetime

Lampiran

- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub

Documentations & Script



**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

4. Data Cleaning (For Data “Pelanggan”)

4.1 Formating Concistency

```
# Cek Konsistensi Format City
cust['city'].value_counts(dropna=False)
```

city	count
Bekasi	533
Jakarta Selatan	473
Depok	404
Surabaya	385
Jakarta Barat	370
Tangerang	366
Bandung	357
Tangerang Selatan	340
Medan	333
Semarang	308
Bogor	307
Jakarta Timur	306
Malang	244
Makassar	242



```
#. Cleaning Data "City"

city_mapping = {
    'Jaktim': 'Jakarta Timur',
    'jakarta timur': 'Jakarta Timur',

    'Jaksel': 'Jakarta Selatan',
    'jakarta selatan': 'Jakarta Selatan',
    'JAKARTA SELATAN': 'Jakarta Selatan',
    'Jakarta selatan': 'Jakarta Selatan',

    'Jakbar': 'Jakarta Barat',
    'jakarta barat': 'Jakarta Barat',
    'JAKARTA BARAT': 'Jakarta Barat',

    'surabaya': 'Surabaya',
    'SBY': 'Surabaya',

    'bandung': 'Bandung',
    'BANDUNG': 'Bandung',

    'TANGSEL': 'Tangerang Selatan',
    'Tangsel': 'Tangerang Selatan',
    'tangerang selatan': 'Tangerang Selatan',

    'Jogja': 'Yogyakarta',
    'Jogjakarta': 'Yogyakarta',
    'yogyakarta': 'Yogyakarta',
    'DIY': 'Yogyakarta'
}

cust['city'] = cust['city'].replace(city_mapping)
```

```
# Cek Hasil Cleaning City
cust['city'].value_counts()
```

city	count
Jakarta Selatan	686
Bekasi	529
Jakarta Barat	529
Surabaya	512
Tangerang Selatan	481
Jakarta Timur	456
Bandung	450
Depok	403
Tangerang	364
Medan	333
Semarang	308
Bogor	305
Malang	244
Makassar	242
Yogyakarta	239
Denpasar	232
Balikpapan	193
Palembang	185

```
# Cleaning City
cust['city'] = cust['city'].str.strip()

# Cek ulang Data
for city in cust['city'].unique():
    print(repr(city))

'Bandung'
'Yogyakarta'
'Tangerang'
'Jakarta Selatan'
'Bandung'
'Malang'
'Bekasi'
'Surabaya'
'Jakarta Barat'
'Tangerang Selatan'
'Jakarta Timur'
'Semarang'
'Palembang'
'Denpasar'
'Medan'
'Makassar'
'Depok'
'Bogor'
'Balikpapan'
```

Problems:

- Inconsistent formatting

Solutions :

- Standarize the formats
- Remove the space before and after to prevent duplicates data.

Lampiran

- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub

Documentations & Script



**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

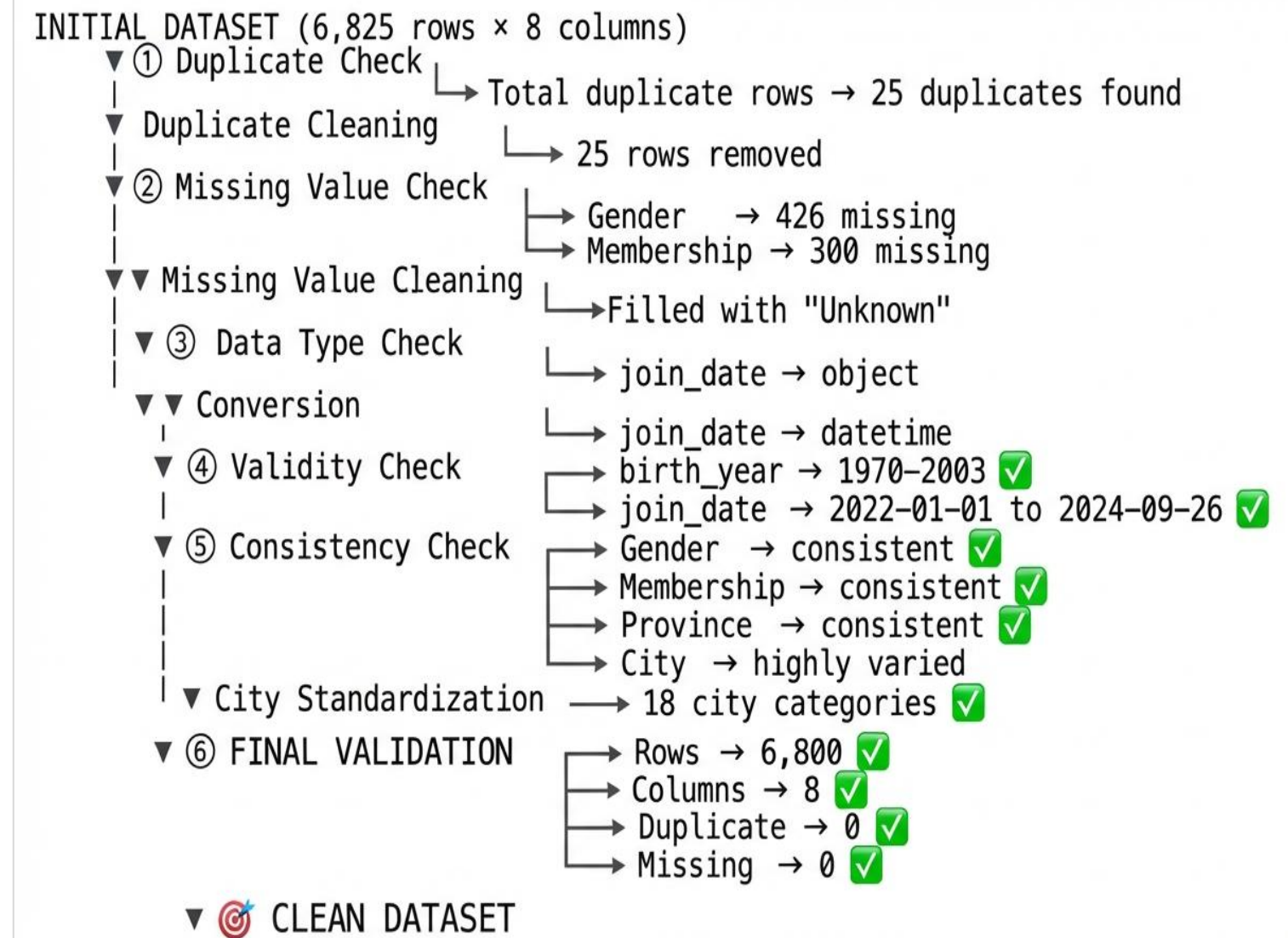
4. Validation Data After Cleaning

```
# Final Validation
print("Jumlah baris:", len(cust))
print("Jumlah kolom:", len(cust.columns))
print("Duplicate:", cust.duplicated().sum())
print("Total missing value:", cust.isna().sum().sum())

[58]
...  Jumlah baris: 6800
      Jumlah kolom: 8
      Duplicate: 0
      Total missing value: 0
```

Notes :

- All steps are repeated for the other datasets.”



- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub



5. Merge & Validasi Data After Merge

```
# Cek Kode Unik
print("Duplicate customer_id:", cust['customer_id'].duplicated().sum())
print("Duplicate product_id:", prod['product_id'].duplicated().sum())
print("Duplicate store_id:", store['store_id'].duplicated().sum())
```

```
Duplicate customer_id: 0
Duplicate product_id: 0
Duplicate store_id: 0
```

```
#. Merge Data Transaksi dengan Pelanggan
df = trans.merge(
    cust,
    on='customer_id',
    how='left',
    suffixes=('', '_customer')
)

print(df.shape)
```

```
(11984, 19)
```

```
# Merge dengan Produk
df = df.merge(
    prod,
    on='product_id',
    how='left',
    suffixes=('', '_product')
)

print(df.shape)
```

```
(11984, 25)
```

```
# Merge dengan Toko
df = df.merge(
    store,
    on='store_id',
    how='left',
    suffixes=('', '_store')
)

print(df.shape)
```

```
(11984, 30)
```

```
#Validasi Hasil Merge
print("Jumlah baris:", len(df))
print("Duplicate order_id:", df['order_id'].duplicated().sum())
print("Store Unknown:", (df['store_id'] == 'Unknown').sum())
```

```
Jumlah baris: 11984
Duplicate order_id: 0
Store Unknown: 240
```


Lampiran

- Documentations
- Script
- Visualization
- Analysis
- Key Insight
- Link GitHub

Documentations & Script



**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

5. Analysis Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# =====
# 1. PREPARE DATA ANALYSIS
# =====

analysis = df[df['order_status'].str.lower() == 'selesai'].copy()

# Metric utama
analysis['gross_sales'] = analysis['qty'] * analysis['unit_price']

analysis['discount_amount'] = (
    analysis['gross_sales'] * analysis['discount_pct'] / 100
)

analysis['net_sales'] = (
    analysis['gross_sales'] - analysis['discount_amount']
)

analysis['cogs'] = (
    analysis['qty'] * analysis['cost_price']
)

analysis['profit'] = (
    analysis['net_sales']
    - analysis['cogs']
    - analysis['shipping_cost']
)

analysis['margin_pct'] = np.where(
    analysis['net_sales'] != 0,
```

Lampiran

- Documentations
- Script
- Visualization
- Analysis Data
- Key Insight
- Link GitHub

Documentations & Script



Seara, Menemani Perjalanan
Datamu Lebih Terarah.

5. Analysis Data (Net Sales, Cogs, Profit & Summary Kuartal)

```
#. Net Sales, cogs dan Profit

df['net_sales'] = (
    df['qty']
    * df['unit_price']
    * (1 - df['discount_pct'] / 100)
)

df['cogs'] = (
    df['qty']
    * df['cost_price']
)

df['profit'] = (
    df['net_sales']
    - df['cogs']
    - df['shipping_cost']
)
```

```
# Cek data
df[['net_sales', 'cogs', 'shipping_cost', 'profit']].head()
```

	net_sales	cogs	shipping_cost	profit
0	1394250.0	1772000.0	75000.0	-452750.0
1	779000.0	470000.0	0.0	309000.0
2	3944400.0	2892000.0	285000.0	767400.0
3	1980000.0	1460000.0	221000.0	299000.0

```
# Summary Kuartal
df_q = df[df['order_status'] == 'Selesai'].copy()

df_q['quarter'] = (
    df_q['order_date']
    .dt.to_period('Q')
    .astype(str)
)

quarter_summary = (
    df_q.groupby('quarter')
    .agg(
        net_sales=('net_sales', 'sum'),
        profit=('profit', 'sum'),
        qty=('qty', 'sum')
    )
    .reset_index()
)

quarter_summary
```

Lampiran

- Documentations
- Script
- Visualization
- Analysis Data
- Key Insight
- Link GitHub

Documentations & Script



Seara, Menemani Perjalanan
Datamu Lebih Terarah.

5. Analysis Data (Summary Kuartal & Profit Change)

```
# Besaran Penurunan Laba
quarter_summary['profit_change'] = (
    quarter_summary['profit'].diff()
)

quarter_summary['profit_change_pct'] = (
    quarter_summary['profit'].pct_change() * 100
)

quarter_summary
```

	quarter	net_sales	profit	qty	profit_change	profit_change_pct
0	2024Q1	4.388098e+09	1.087881e+09	1521	NaN	NaN
1	2024Q2	5.310364e+09	1.403834e+09	1825	3.159530e+08	29.042966
2	2024Q3	5.089994e+09	1.961607e+09	1446	5.577725e+08	39.732075
3	2024Q4	5.369132e+09	1.094642e+09	1994	-8.669649e+08	-44.196674
4	2025Q1	6.077445e+09	1.556784e+09	2076	4.621419e+08	42.218554

```
#. Kuartal dengan penurunan Terbesar
decline = quarter_summary[
    quarter_summary['profit_change'] < 0
].copy()

decline.sort_values('profit_change')
```

	quarter	net_sales	profit	qty	profit_change	profit_change_pct
6	2025Q3	5.293965e+09	1.420858e+09	1754	-1.376693e+09	-49.210646
3	2024Q4	5.369132e+09	1.094642e+09	1994	-8.669649e+08	-44.196674

Lampiran

- Documentations
- Script
- Visualization
- Analysis Data
- Key Insight
- Link GitHub

Documentations & Script

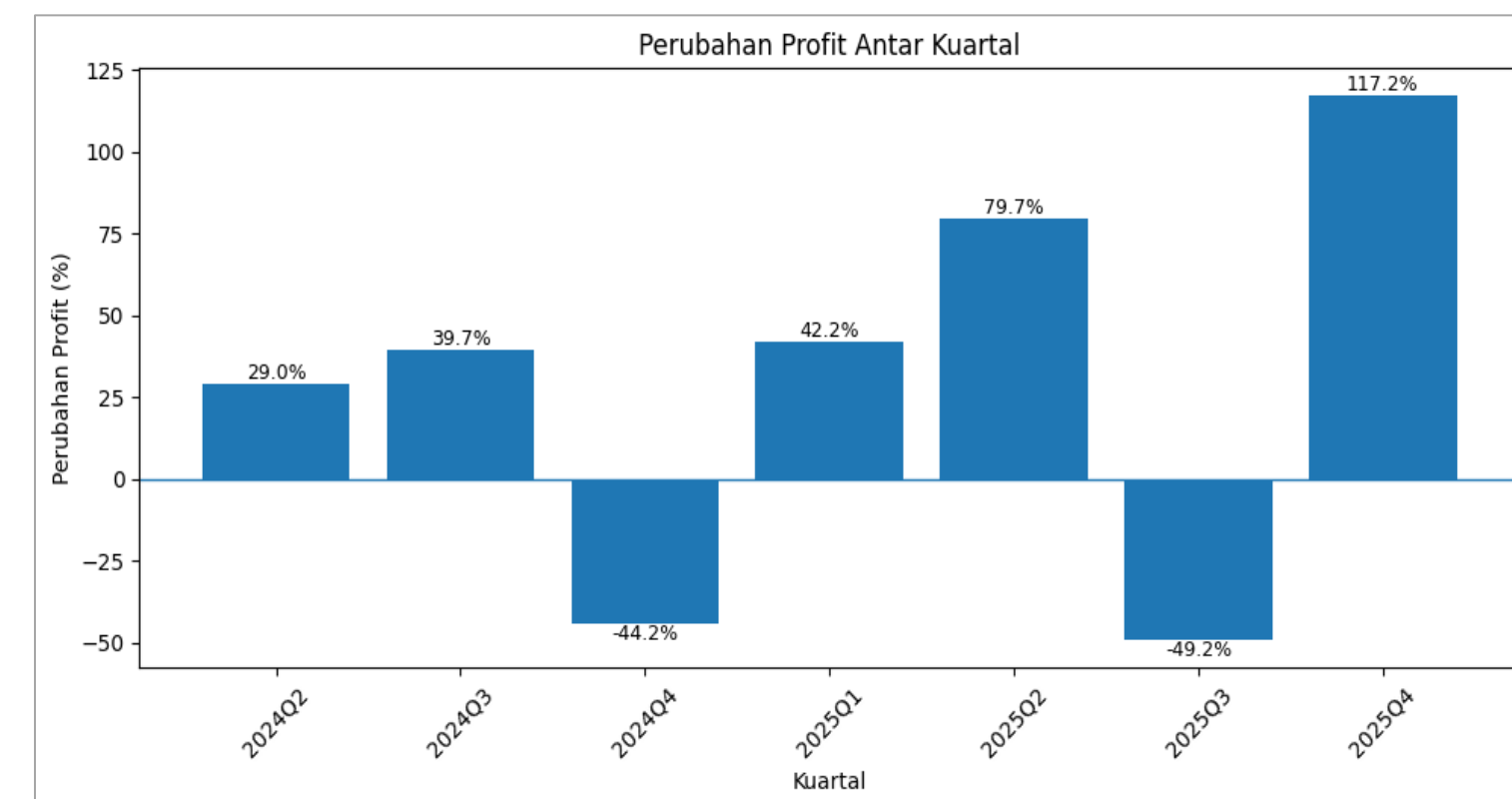
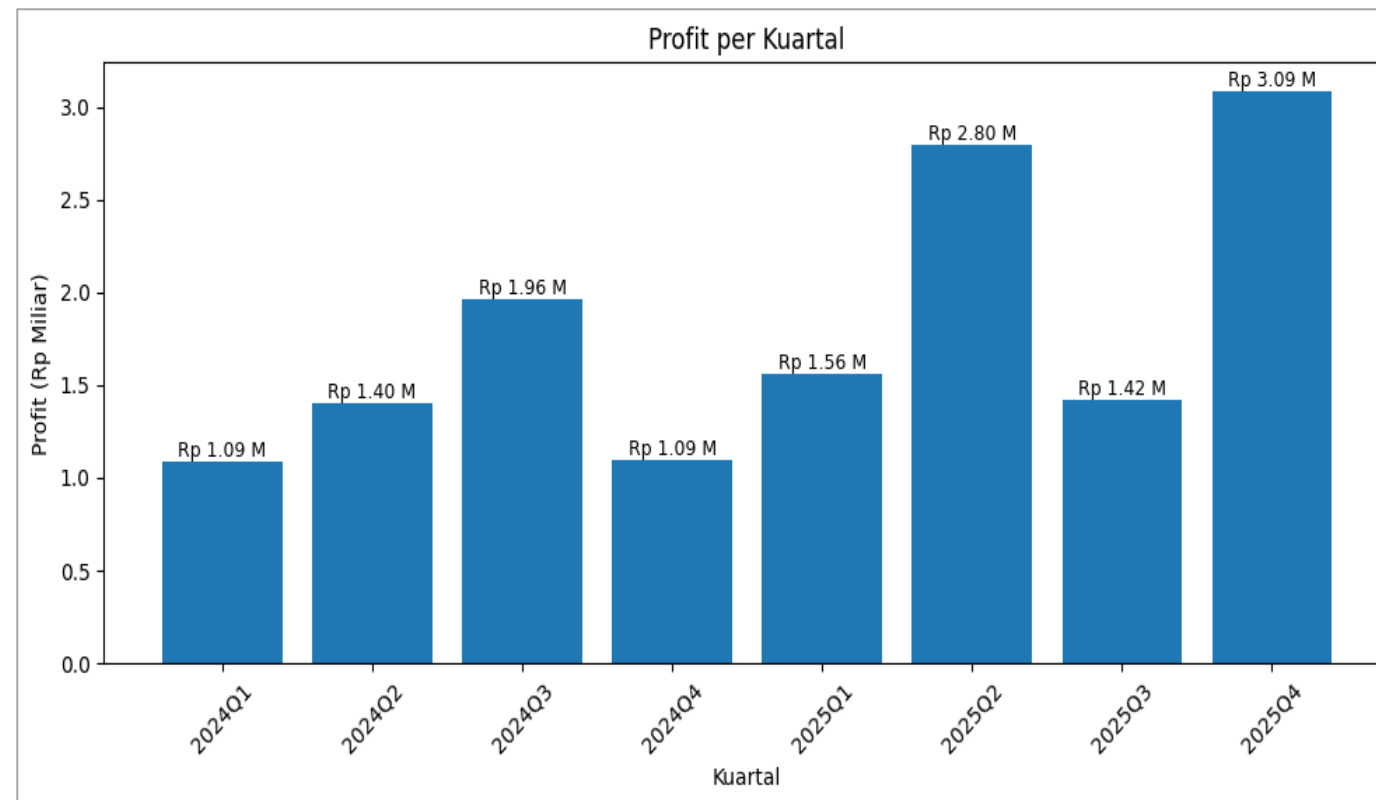


Seara, Menemani Perjalanan
Datamu Lebih Terarah.

6. Data Analysis & Key Insight

Q1

How significant was the decline in profit, and in which quarter exactly did it occur?



A1

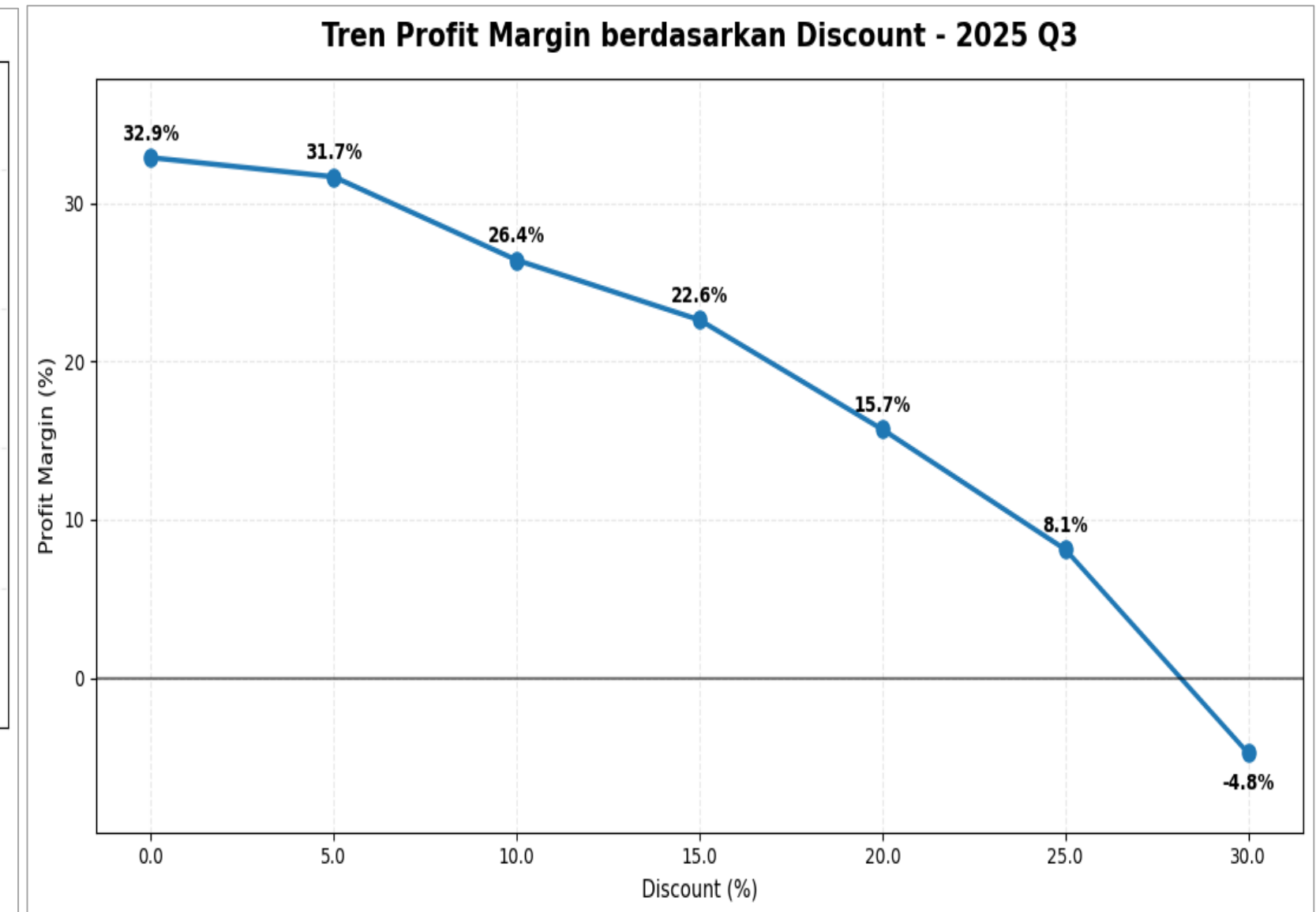
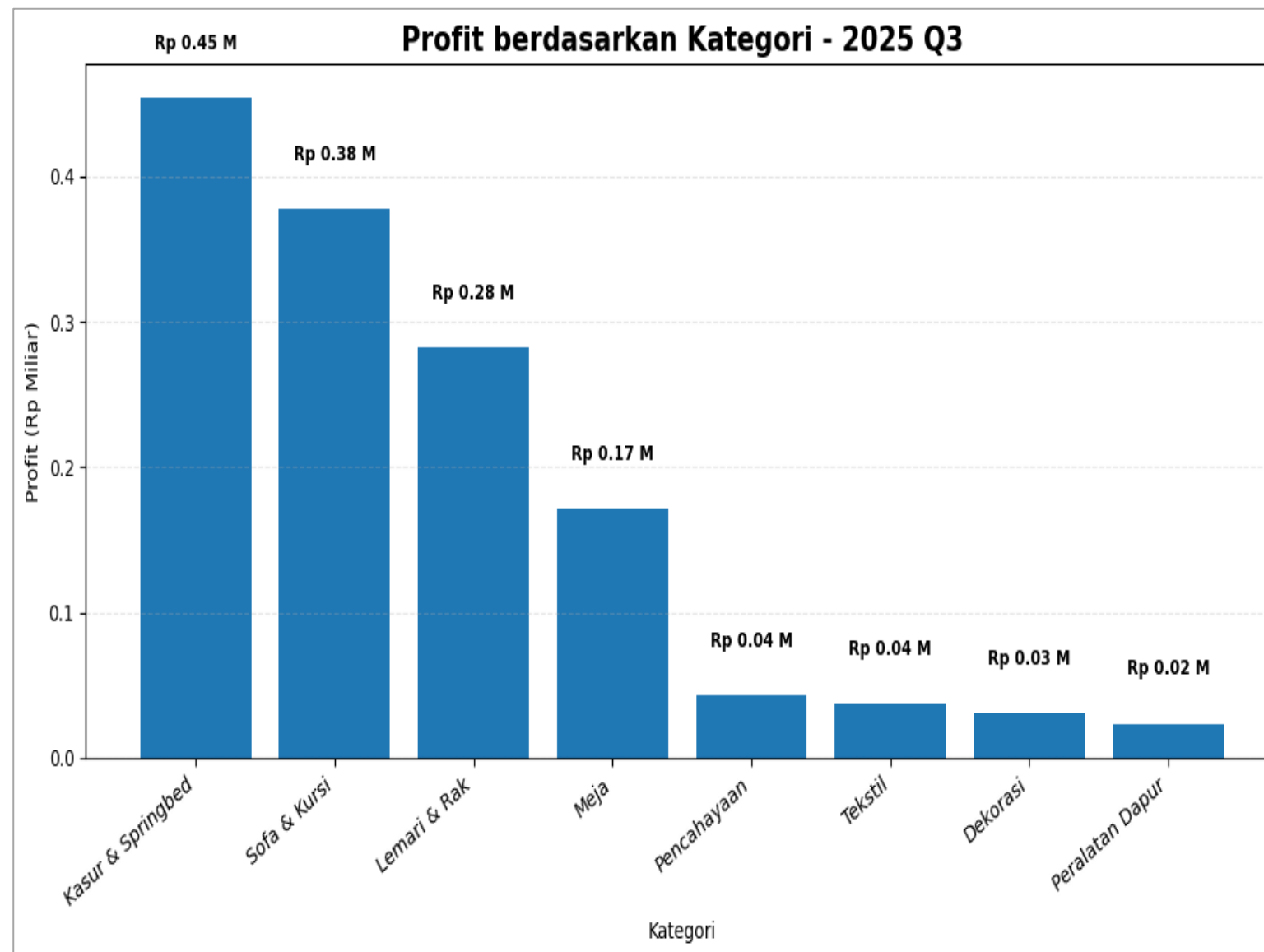
Peak profit hit Rp3.09 billion in 2025Q4. However, the sharpest drop occurred in 2025Q3, plunging by **Rp1.38 billion (49.21%) quarter-over-quarter**. This downturn requires further investigation.



Seara, Menemani Perjalanan
Datamu Lebih Terarah.

Q2

Is it driven by product mix, sales channels, or discount policies?



Lampiran

Documentations

Script

Visualization

Analysis Data

Key Insight

Link GitHub

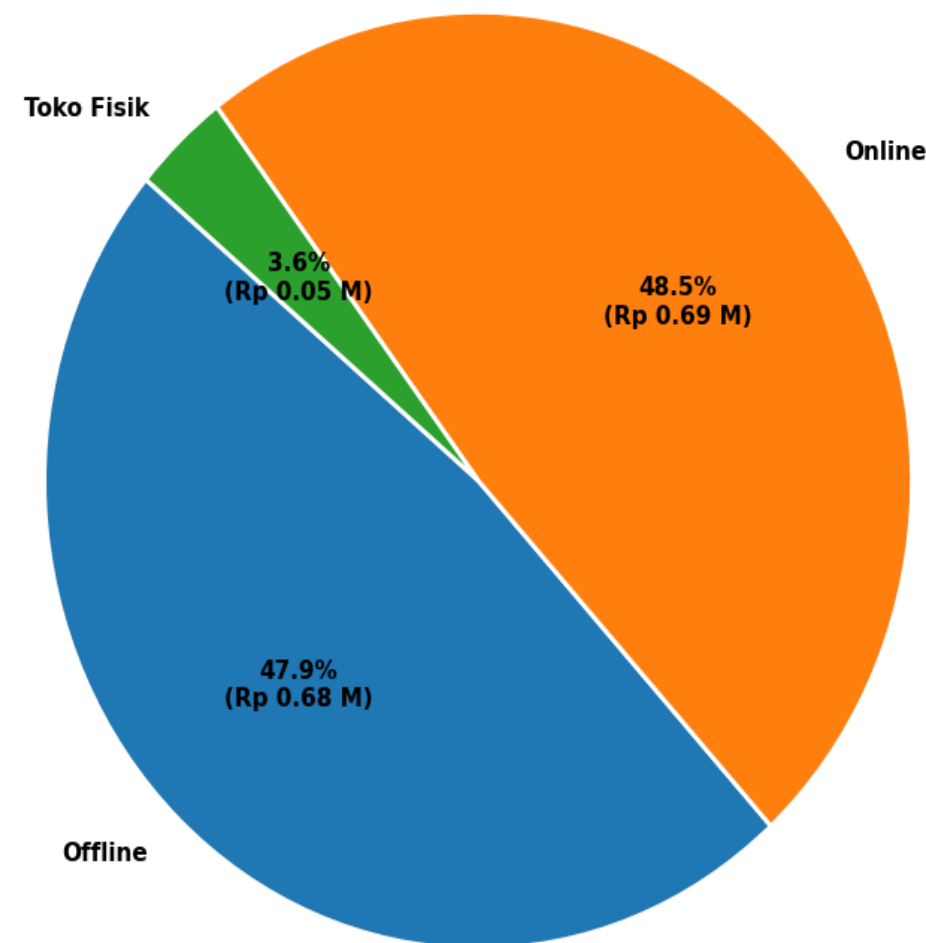


Seara, Menemani Perjalanan
Datamu Lebih Terarah.

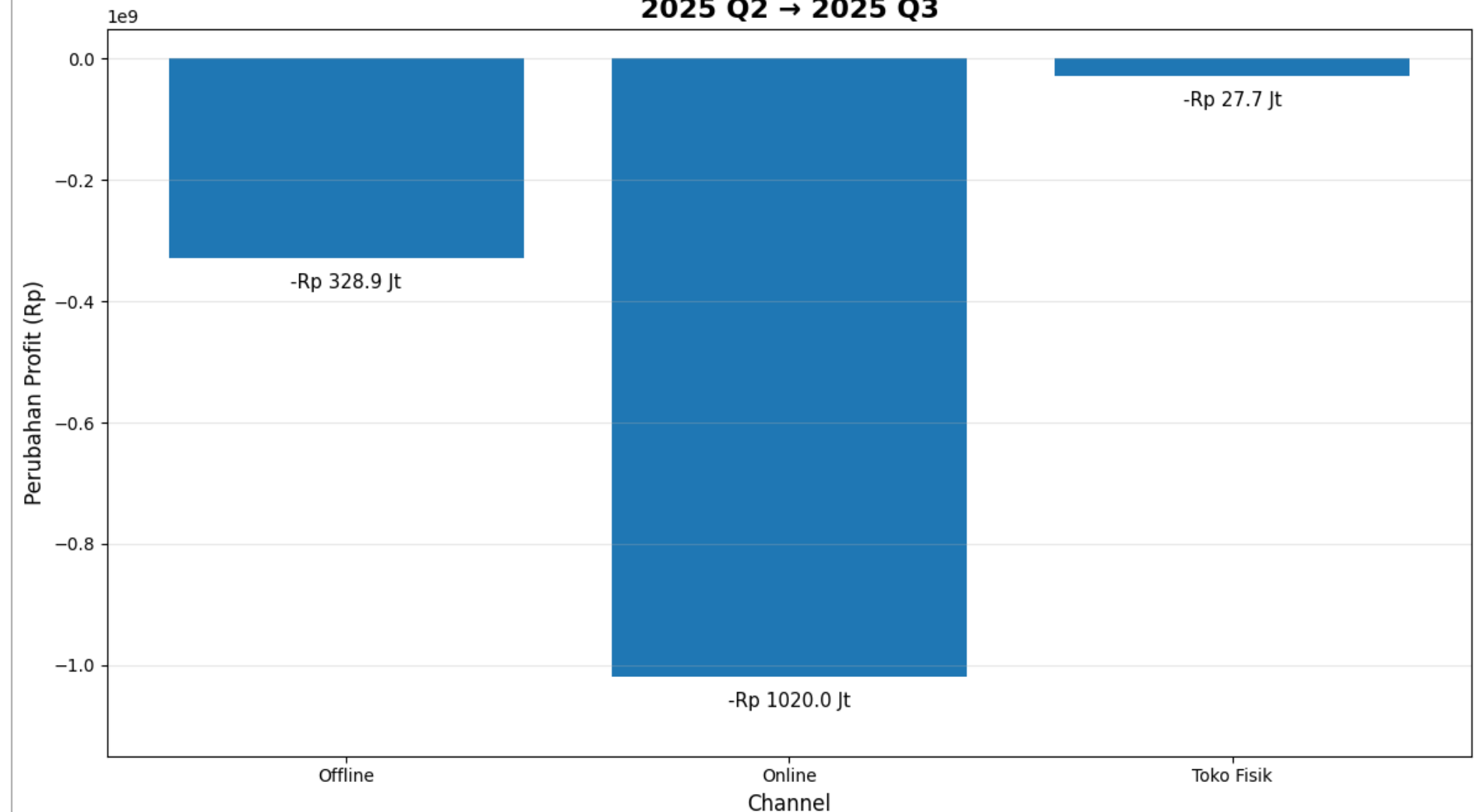
Q2

Is it driven by product mix, sales channels, or discount policies?

Kontribusi Profit berdasarkan Channel - 2025 Q3



Perubahan Profit berdasarkan Channel
2025 Q2 → 2025 Q3



A2

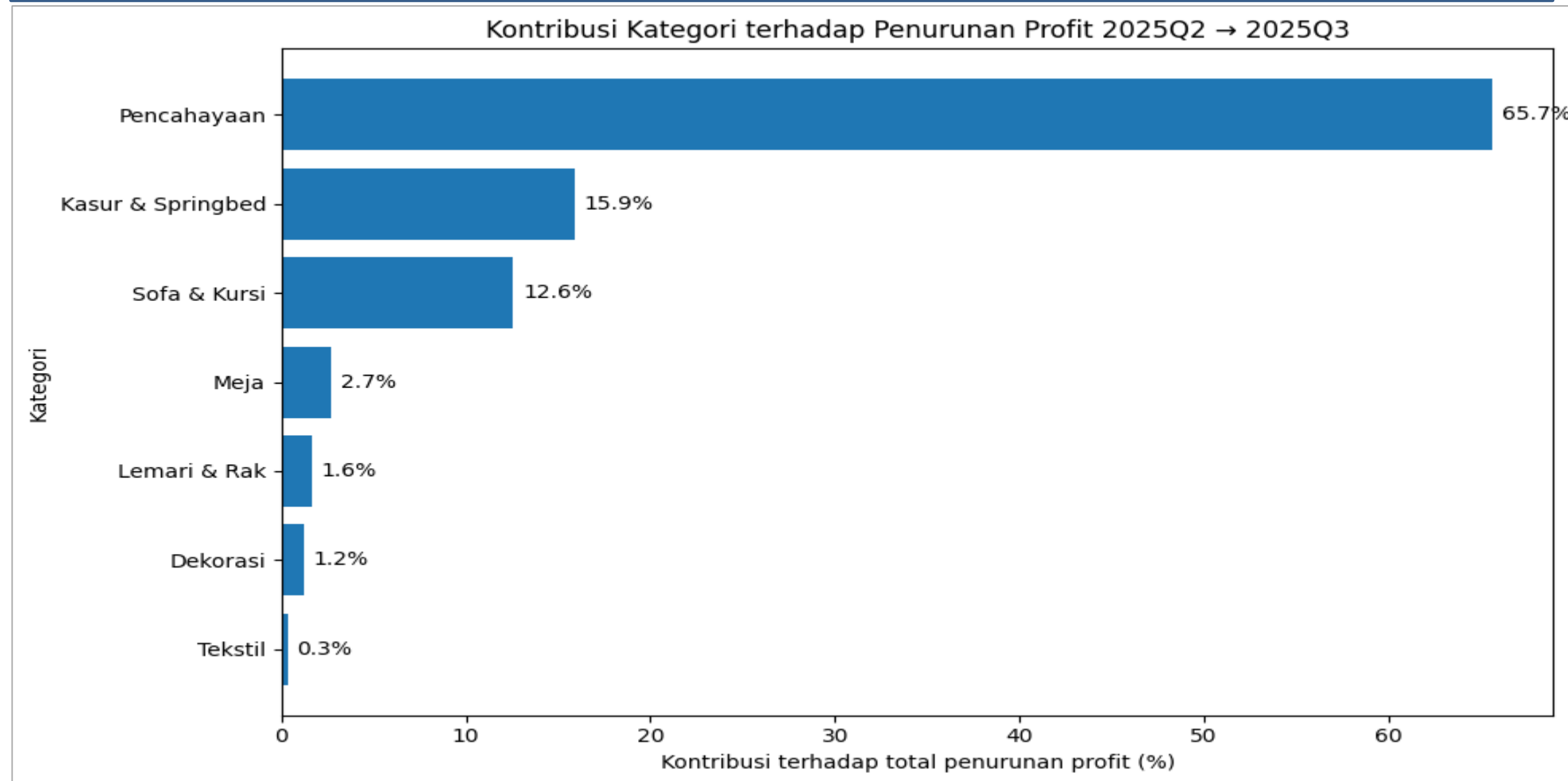
Online was the biggest contributor to the Q3 2025 profit decline. Online profit dropped by **around Rp1.02 billion (-59.7%)**, a much larger nominal decrease compared to Offline and Brick-and-Mortar.



Seara, Menemani Perjalanan
Datamu Lebih Terarah.

Q3

Which category was hit the hardest, and how much did it contribute?



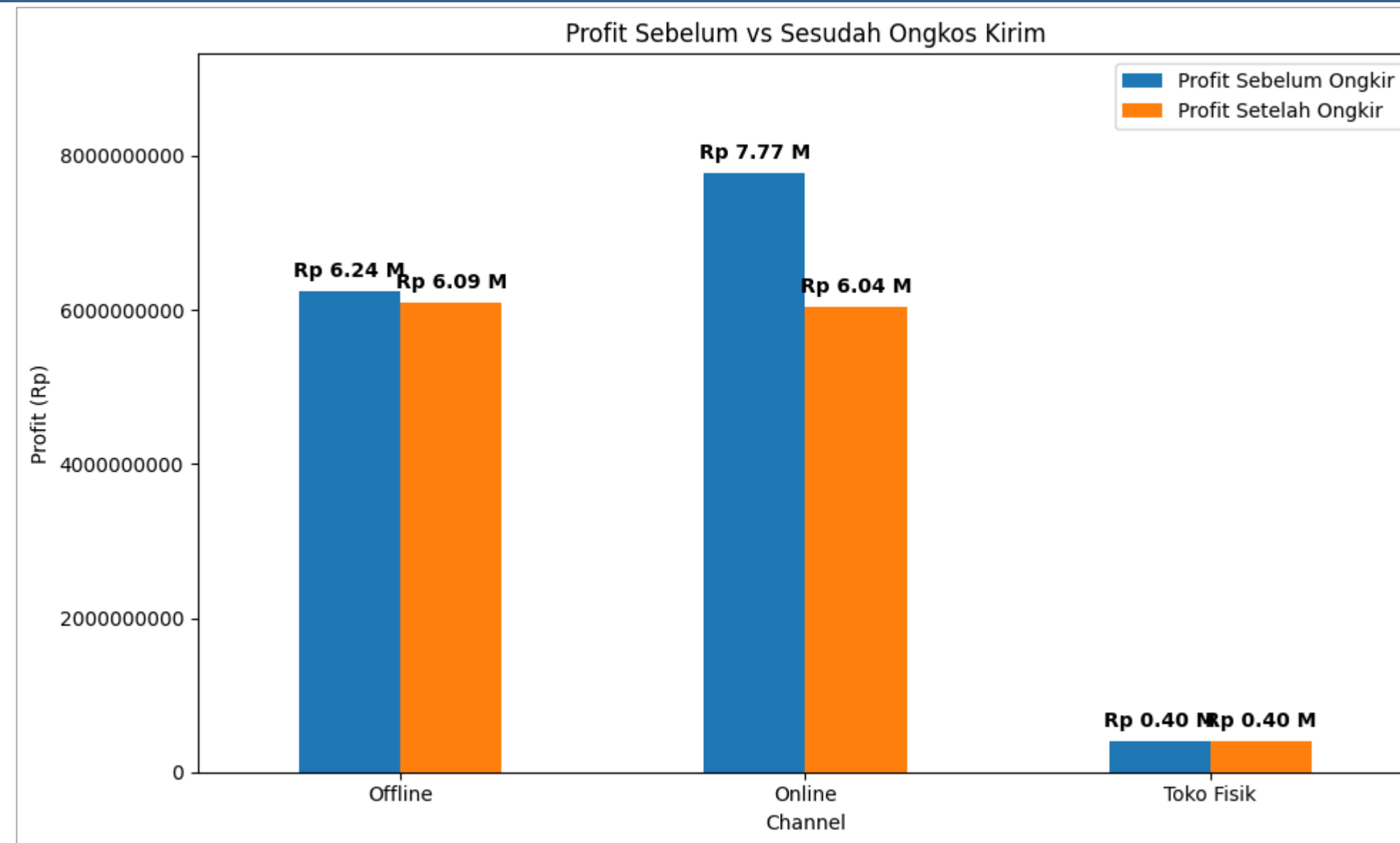
A3

Lighting took the hardest hit, contributing 65.7% to the decline.



Q4

Does online remain profitable once shipping costs are included?



A4

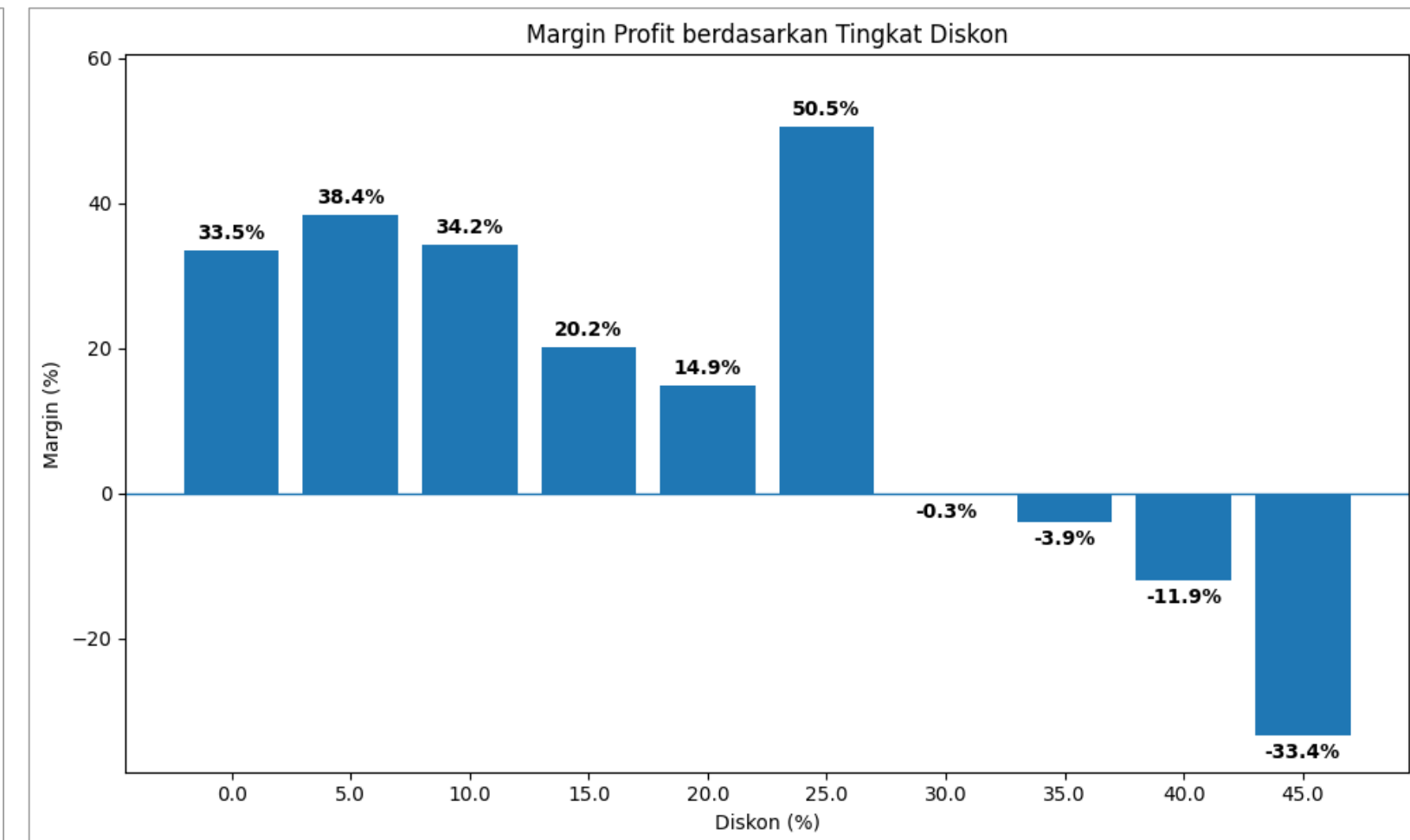
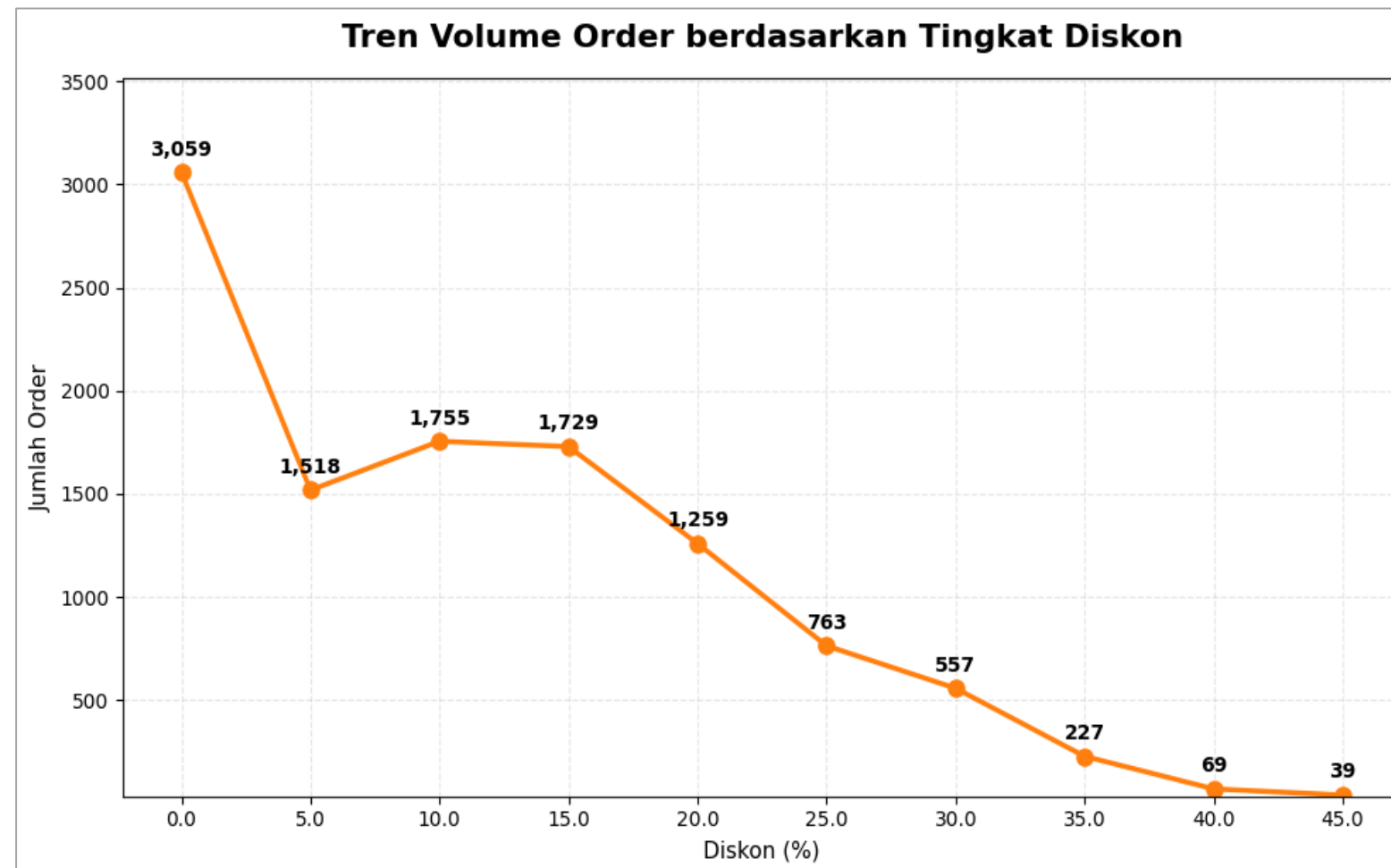
The Online channel remains profitable after shipping costs are accounted for, with a net profit of approximately Rp6.04 billion, despite shipping costs reducing profit by 22.24%.



Seara, Menemani Perjalanan
Datamu Lebih Terarah.

Q5

How can we quickly boost margins without damaging sales volume?



A5

0–20% discount: Volume stays high, but margins start getting squeezed as discounts increase.

30% and above: Margins turn negative, while volume continues to shrink.

Insight :controlling high discounts (especially $\geq 30\%$) is the fastest way to fix margins without completely cutting promo activities.



**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**

Conclusion

Key Takeaways

Through this session, I learned the end-to-end data analyst workflow using Python. First, I focused on **Data Understanding and Data Cleaning** using Pandas and NumPy to inspect the datasets, identify missing values, duplicates, incorrect data types, and potential outliers. Next, I performed **Data Transformation and Analysis** by merging datasets, creating new variables, grouping data, and calculating key metrics such as sales, profit, margin, and changes across quarters. Finally, I used **Data Visualization** with Matplotlib and Seaborn to transform the analysis into clear charts and identify business insights, such as profit decline, category performance, channel profitability, shipping cost impact, and the effect of discount policies. Through this process, I learned how Python can be used to turn raw data into meaningful insights that support data-driven business decisions.



**Seara, Menemani Perjalanan
Datamu Lebih Terarah.**



Let's Connect



 **LinkedIn : Rahmanida Susiana**

 **Email : Rahmanidasusiana@gmail.com**

 **GitHub : <https://github.com/Rahmanida01>**